MediAI

SMART PATIENT TRIAGE & EMERGENCY RISK PREDICTION SYSTEM

Common Project & UI Guidelines

TEAM-WIDE DEVELOPMENT, DESIGN SYSTEM & INTEGRATION RULES

Clinical safety: This is an AI-assisted decision-support prototype. It must not independently diagnose, prescribe, or replace certified clinical judgment.

Prepared for coordinated module development and integration.

Purpose

This document is shared by all five members. Module pages may look different when their workflows differ, but the technical contracts, accessibility rules, security boundaries and semantic design tokens must remain consistent.

1. Repository and branch governance

- One shared GitHub repository. main is the stable integration branch; each member works only on their assigned feature branch.
- Create feature branches from the current approved main; keep changes focused, commit frequently, push branch and open a Pull Request. Do not push directly to main.
- Shared files (package.json, global styles/tokens, app layout, shared components, database migrations, API contracts) require team agreement and review.
- Before merging: pull/rebase current main as team policy allows, run lint/typecheck/tests, resolve conflicts locally and verify the full application.

2. Technology baseline

- Frontend/framework: React + Next.js App Router and TypeScript (project-agreed stack).
- Backend: Next.js Route Handlers/API routes; business rules live in server-side application/domain services, not UI components.
- Database: Supabase PostgreSQL with versioned migrations and Row-Level Security (RLS).
- Authentication: Supabase Auth plus server-side role/ownership authorization.
- AI: specified ML ensemble + TreeSHAP explainability. If Gemini is used for auxiliary language features, keep calls server-side and never use it to replace deterministic safety rules or required ML/XAI behavior.
- Version control: Git and GitHub. UI generation: each member may use Antigravity, but all generated code must obey the same repository contracts.

3. Shared UI/design system

- Design freedom: each module may choose page layout suited to its user workflow; consistency is required in design tokens, typography, spacing scale, focus states, form behavior, navigation conventions and component accessibility.
- Use one shared source of truth for color tokens and common components (Button, Input, Select, Card, Badge, Alert, Modal, LoadingState, EmptyState, ErrorState). Avoid creating duplicate global components inside module folders.
- Semantic acuity colors are fixed by the master spec: Critical #E53E3E, High #ED8936, Moderate #ECC94B, Low #38A169. Always show text/numeric labels as well as colors.
- Meet WCAG AA contrast (>4.5:1), keyboard accessibility, visible focus, responsive layouts and accessible labels.
- English/Tamil localization must support en-US and ta-IN; avoid hardcoding user-facing strings directly in components.

4. Architecture boundaries

- Follow Presentation → Application → Domain/Business → Data Access → Database. Dependencies flow downward only.
- UI components must not contain clinical rules, direct SQL, or persistence queries.
- External speech/notification providers must be wrapped in adapters.
- Never weaken or bypass red-flag safety logic. A Tier-1 red flag forces Critical/P1 and bypasses ML inference.
- Clinical decision-support only: no definitive diagnosis, automated prescription, dosage calculation, dispensing, or replacement of certified clinical judgment.

5. API and data contracts

- Use the standard success envelope: {data: {}, meta: {}, error: null}.
- Use the standard error envelope: {data: null, meta: {}, error: {code: "CODE", message: "Human message", details: []}}.
- Master-defined endpoints: POST /api/v1/triage/assess; GET /api/v1/doctor/queue?status=PENDING_REVIEW; POST /api/v1/doctor/override.
- Use UUID patient identifiers server-side; never trust a client-supplied patient_id without ownership/role authorization.
- Do not change schema fields, enum values, endpoint names or payload contracts unilaterally. Propose changes in a PR and update migration/API docs with team approval.

6. Security and PHI

- Never expose Supabase service-role keys, Gemini API keys or other secrets in client bundles. Use server-side environment variables and commit only .env.example placeholders.
- Enforce least privilege and RBAC on every server endpoint; UI route guards are not security boundaries.
- Apply RLS and server-side ownership checks. No plaintext PHI in URLs, browser console, analytics, error messages, mock fixtures or source control.
- Master spec calls for AES-256-GCM at rest, TLS 1.3 in transit, encrypted name field, de-identification for ML and immutable append-only audit logging. Validate implementation constraints with the actual hosting environment.
- System Administrator has zero PHI visibility; patient identifiers are masked with SHA-256 in admin views as specified.

7. Git/Antigravity working agreement

- Each member opens Antigravity against the same cloned repository and checks out their own branch before asking the agent to edit files.
- Use the Read-First Mandate: inspect README, docs, current architecture, types, schema and relevant module code before generating changes.
- Tell Antigravity the exact assigned module, allowed directories, API/schema contracts and forbidden shared files. Ask it to list planned file changes before broad refactors.
- Do not let an agent rewrite package.json, global layouts, migrations or shared API types without team approval.
- After code generation, inspect diffs, run lint/typecheck/tests and manually review security-sensitive code. Never accept fabricated test results.

8. Integration and release gates

- Integration order recommendation: common scaffold/contracts → M1 identity → M2 intake/vitals → M4 safety orchestration with M3 inference integration → M5 doctor workstation; parallel UI work can continue using approved mocks.
- Use typed shared DTOs and API contract tests; mocks must match the real response envelope.
- Release gates: red-flag safety tests, RBAC/RLS tests, API schema tests, full intake-to-override E2E workflow, no secrets/PHI leaks, WCAG AA checks and responsive tests.
- Master definition of done: acceptance criteria met, zero regressions on safety tests, standardized JSON envelopes, server-side RBAC, zero raw PHI exposure, WCAG AA compliance.

9. Open decisions to confirm before coding

- The master spec names PostgreSQL DDL and a specific directory architecture; team-selected Supabase/Next.js implementation must preserve these contracts and map them into the approved Next.js directory structure.
- Patient CRUD/detail endpoints, analytics endpoints, medication schema, notification provider and exact ML model artifact/registry schema are not fully specified in the master PDF. The team must agree these before implementation; do not fabricate them.

- The project member allocation includes medication management, but the master scope explicitly excludes prescribing, dosage calculation and dispensing. Limit medication UI to approved record/reconciliation use unless the master scope is formally revised.
- Clinical thresholds and red-flag definitions must be reviewed by a qualified clinical supervisor before any real-world use. This is a prototype, not a validated clinical device.

Team and branch ownership

Member	Module	Branch
Jesica Jasmine	M1: Patient Management & Authentication	feature/m1-patient-auth
Joselin Rubha	M2: Health Assessment & Medical History	feature/m2-health-assessment
Madhu Mitha	M3: AI Risk Prediction & Explainability	feature/m3-ai-prediction
Raga Priya	M4: Smart Triage, Safety Rules & Alerts	feature/m4-smart-triage
Bhuvana	M5: Doctor Dashboard, Medication Management & Analytics	feature/m5-doctor-dashboard

Shared API envelope examples

Success

```
{"data": {}, "meta": {}, "error": null}
```

Error

```
{"data": null, "meta": {}, "error": {"code": "CODE", "message": "Human message", "details": []}}
```

Source alignment note

This team pack is derived from MediAI Master Project Specification v2.0 and the stack/branch choices agreed in the conversation. Where the master file does not define a detail (for example patient CRUD endpoint names, analytics APIs, medication schema, notification provider or model registry), this pack explicitly marks it for team approval rather than inventing a contract.